AGENTGUARD · AI AGENT SECURITY & GOVERNANCE

Deploying AgentGuard

Running the platform and its services locally and in the cloud

Generated from `docs/DEPLOYMENT.md`

How to run the platform and its services **locally** and **in the cloud**.

This is the deployment companion to `RUNNING.md` (day-to-day dev, config reference, testing) and `ARCHITECTURE.md` (why the system is shaped this way). Where this guide says "see RUNNING.md §N", that section has the exhaustive detail; this guide is the end-to-end path from a clean machine (or a clean cloud account) to a working deployment.

---

1. The shape of a deployment

One AgentGuard deployment is **one region**. Everything below is what goes into that one region; §7 explains standing up more than one.

Tier	What it is	Image	State	Scaling
`web`	Next.js 14 dashboard	`apps/web/Dockerfile`	none	horizontal, cheap
`api`	FastAPI control-plane **and** the runtime decision endpoint	`apps/api/Dockerfile`	none (Redis holds sessions, rate limits, approvals, policy cache)	horizontal — this is the tier that matters
`ai-analyst`	read-only natural-language Q&A service	same image as `api`, entrypoint `agentguard_api.analyst.asgi:app`	none	horizontal, independent of `api`
`migrate`	one-shot: `alembic upgrade head` + RBAC seed + ClickHouse schema	same image as `api`	—	runs to completion, then exits
`workers`	event-stream tier (ClickHouse baselines, campaign detection) — **scaffolded**; the synchronous detection / risk / DLP paths run in-process in `api` today	same image as `api`	none	horizontal (when populated)

Backing services (stateful — one set per region)

Service	Role	Local (Compose)	Cloud (managed equivalent)
PostgreSQL 16 + `pgvector`	source of truth (46 tables)	`pgvector/pgvector:pg16`	RDS / Aurora PostgreSQL · Cloud SQL · Azure Database for PostgreSQL
Redis 7	cache, rate limits, session + approval state, usage counters	`redis:7-alpine`	ElastiCache · Memorystore · Azure Cache for Redis
Redpanda / Kafka	runtime event transport	`redpandadata/redpanda`	MSK · Confluent Cloud · Redpanda Cloud · Event Hubs (Kafka API)
ClickHouse	high-volume event store + analytics	`clickhouse/clickhouse-server:24.8`	ClickHouse Cloud · self-managed on nodes
Qdrant	threat-intel / attack-pattern vectors (**never** authorization)	`qdrant/qdrant`	Qdrant Cloud · self-managed
S3-compatible object store	red-team evidence, exports, large payloads	`minio/minio`	S3 · GCS · Azure Blob (S3 API)

The `api` image needs **only** Postgres and Redis to serve traffic (`/readyz` checks exactly those two). Kafka, ClickHouse, Qdrant and object storage are used by best-effort side-channels and the security/analytics planes — a runtime decision never blocks on them.

---

2. Configuration model

Every setting is an environment variable read by `apps/api/agentguard_api/config.py` (prefix `AGENTGUARD_`) or a datastore URL. **Every value has a working default** tuned for local Compose; production overrides a known short list. Full reference: `RUNNING.md` §4.

The variables that **must** change for any real deployment:

Var	Why
`AGENTGUARD_ENV`	set to `production` — tightens error responses, disables dev niceties
`AGENTGUARD_SECRET_KEY`	signs JWTs and state tokens. **Unique per region.** `openssl rand -hex 32`
`AGENTGUARD_OPEN_REGISTRATION`	`false` once the first org exists
`DATABASE_URL`	`postgresql+asyncpg://USER:PASS@HOST:5432/DB`
`REDIS_URL`	`redis://[:PASS@]HOST:6379/0` (use `rediss://` for TLS)
`AGENTGUARD_CORS_ORIGINS`	the dashboard's public origin(s), comma-separated
`AGENTGUARD_OAUTH_REDIRECT_BASE`	public base URL of `api` (e.g. `https://us.api.agentguard.example`)
`AGENTGUARD_WEB_BASE_URL`	public base URL of `web`
`NEXT_PUBLIC_API_BASE_URL`	**build-time** for `web` — the browser's API origin
`AGENTGUARD_REGION` / `AGENTGUARD_REGIONS`	see §7

Optional, enabled only when set: `ANTHROPIC_API_KEY` (analyst uses Claude instead of the deterministic router), `GOOGLE_CLIENT_ID` / `_SECRET`, `MICROSOFT_CLIENT_ID` / `_SECRET` / `_TENANT`. Enterprise SAML/OIDC and SCIM are configured **per-org through the API**, never by env.

> `NEXT_PUBLIC_API_BASE_URL` is baked into the web bundle at `npm run build`. Changing the API origin means rebuilding the `web` image.

---

3. Local — everything in containers

The fastest path. Needs only Docker + Docker Compose v2.

```
git clone <repo> && cd agent-guard
cp .env.example .env
make up            # docker compose -f infra/docker-compose.yml --profile apps up --build -d
#  Windows:  .\tasks.ps1 up
```

`make up` builds the images, starts the six backing services, runs the `migrate` container to completion (migrations + RBAC seed + ClickHouse schema), then starts `api`, `ai-analyst`, and `web`.

URL	What
http://localhost:3010	dashboard — use the **Create account** tab for the first org
http://localhost:8010/docs	API — interactive OpenAPI
http://localhost:8010/healthz · `/readyz`	probes
http://localhost:8020/docs	AI-analyst service

Host ports come from `.env` (`API_PORT`, `WEB_PORT`, …) and sit on an AgentGuard-specific block (54xx / 63xx / 90xx) so they don't collide with other local stacks. Without a `.env`, Compose falls back to 8000 / 3000 / 8020.

```
make infra-ps      # status          make down          # stop app + infra
make infra-logs    # tail logs       make infra-clean   # stop + delete volumes (wipes data)
```

---

4. Local — native app, containerised infra (for development)

Run Postgres/Redis/etc. in Compose but the API and web on the host with hot-reload. This is the normal inner-loop setup — full steps in `RUNNING.md` §5. In short:

```
cp .env.example .env
make infra-up                    # backing services only
make api-install                 # venv + editable installs (policy-engine, sdk-python,
api[dev])
make db-migrate db-seed events-migrate
make api-dev                     # uvicorn --reload --port 8010
make web-install web-dev         # next dev, :3010   (separate terminal)
```

---

5. Cloud — build and publish images

Three images, all built from the repo root as context:

```
# API (also the ai-analyst and migrate image — same artifact, different command)
docker build -f apps/api/Dockerfile -t $REGISTRY/agentguard-api:$TAG .

# Web — build context is apps/web, and the API origin is baked in here
docker build -f apps/web/Dockerfile \
  --build-arg NEXT_PUBLIC_API_BASE_URL=https://us.api.agentguard.example \
  -t $REGISTRY/agentguard-web:$TAG apps/web
```

> The committed `apps/web/Dockerfile` reads `NEXT_PUBLIC_API_BASE_URL` from the environment at build time. If your builder doesn't forward `--build-arg` into the build environment, add an `ARG`

> `NEXT_PUBLIC_API_BASE_URL` / `ENV NEXT_PUBLIC_API_BASE_URL=$NEXT_PUBLIC_API_BASE_URL` pair before `RUN npm run build`.

Push both tags to your registry (ECR / Artifact Registry / ACR / GHCR). Pin a digest for production rollouts, not a moving tag.

The API image already declares a `HEALTHCHECK` (`/healthz`) and runs `uvicorn agentguard_api.main:app` on `:8000`. The web image runs the Next.js standalone server on `:3000`.

---

6. Cloud — Kubernetes (the reference target)

PRD §55–56 targets **EKS**. `infra/k8s/` reserves the namespace layout; the manifests below are the deployment shape to fill in.

```
agentguard-api        de:  api (N replicas, HPA)   svc:  ClusterIP :8000
                      de:  ai-analyst (M replicas, HPA)
agentguard-security   (red-team / threat workers — later)
agentguard-workers    de:  workers  (event-stream tier — later)
agentguard-data       StatefulSets OR ExternalName to managed services
monitoring            Prometheus / Grafana / OTel collector
```

6.1 Order of operations

1. **Secrets** — create a `Secret` (or wire External Secrets / CSI to your secret manager) holding `AGENTGUARD_SECRET_KEY`, `DATABASE_URL`, `REDIS_URL`, `S3_*`, `ANTHROPIC_API_KEY`, IdP client secrets. Non-secret config goes in a `ConfigMap` (`AGENTGUARD_ENV`, `AGENTGUARD_REGION`, `AGENTGUARD_REGIONS`, `AGENTGUARD_CORS_ORIGINS`, the two base-URL vars, `CLICKHOUSE_HOST`, `KAFKA_BOOTSTRAP_SERVERS`, `QDRANT_URL`).
2. **Migrations** — run as a `Job` (or an `argo`/Helm pre-install hook) using the API image: `command: ["/bin/sh","-c", "alembic upgrade head && python -m agentguard_api.rbac.seed && python -m agentguard_api.events.migrate"]` Gate the app rollout on this Job succeeding. It is idempotent — safe to run every deploy.
3. `api` **Deployment** — image `agentguard-api:$TAG`, port `8000`, `envFrom` the ConfigMap + Secret.
   - `livenessProbe: GET /healthz` — never touches dependencies.
   - `readinessProbe: GET /readyz` — 200 only when Postgres **and** Redis are reachable.
   - Resource requests / limits; start at `requests: 250m / 512Mi`.
   - `HorizontalPodAutoscaler` on CPU (target ~65%) — the hot path is CPU-bound and makes no LLM call.
   - `PodDisruptionBudget: minAvailable: 1` (raise for prod).
4. `ai-analyst` **Deployment** — same image, override `command: ["uvicorn","agentguard_api.analyst.asgi:app","--host","0.0.0.0","--port","8000"]`. Separate Deployment, Service, and HPA so a burst of analyst questions never steals capacity from runtime decisions.
5. `web` **Deployment** — image `agentguard-web:$TAG`, port `3000`. Liveness = `GET /` on `:3000`.
6. **Ingress / gateway** — terminate TLS at the edge (ALB / GKE Ingress / App Gateway or an ingress-nginx + cert-manager). Route:

- `app.<region>.agentguard.example` → `web` Service
- `api.<region>.agentguard.example` → `api` Service
- Optionally `analyst.<region>…` → `ai-analyst`, or keep it internal-only (the `api` already serves `/v1/analyst/*` for the dashboard).

7. **NetworkPolicies** — default-deny per namespace; allow `api` → `data`, `web` → `api`, ingress → `web`/`api`.

6.2 Backing services

Point the URLs at **managed services** (see the table in §1) via `ExternalName` Services or straight connection strings in the Secret. Running Postgres/Kafka/ClickHouse yourself as StatefulSets is supported but you own backups, upgrades, and failover. Managed Postgres with PITR is strongly preferred — it is the source of truth.

6.3 Plain-VM / Compose-in-prod

Not the target, but viable for a single-tenant or pilot install: run `infra/docker-compose.yml --profile apps` on one host behind a TLS-terminating reverse proxy, with `.env` supplying production secrets and `restart: unless-stopped` (already set). Move Postgres off-box and back it up.

---

7. Multi-region

Each region is a **complete, isolated stack** — its own Postgres, Redis, ClickHouse, Redpanda, object store, and its own `AGENTGUARD_SECRET_KEY`. There is no cross-region database and no global router in the app tier. An org created in a region is pinned to it forever; a request that reaches the wrong region gets `421 Misdirected Request` with the correct URL in the `X-AgentGuard-Region-Url` header.

To add region `eu`:

1. Provision fresh infrastructure and a fresh `AGENTGUARD_SECRET_KEY` for `eu`.
2. Deploy `api` + `web` + `ai-analyst` there with:
   - `AGENTGUARD_REGION=eu`
   - `AGENTGUARD_REGIONS` listing **every** region, identical string in every deployment: `us|United States|https://us.api…|https://us.app…, eu|European Union|https://eu.api…|https://eu.app…`
   - datastore URLs (`DATABASE_URL`, `REDIS_URL`, `S3_*`, …) pointing at the EU infra
3. Run the migrate Job against the EU database.
4. Point DNS: `eu.api.agentguard.example`, `eu.app.agentguard.example`.

No application code changes. `GET /v1/regions` now advertises both and the dashboard sign-up shows a region picker. Details: `MULTI_REGION.md`.

---

8. Observability

Signal	How	Where to send it
Logs	structlog JSON on stdout	container log pipeline → CloudWatch / Cloud Logging / Loki
Metrics	scrape `api` / `web` pods; watch p95 of `POST /v1/runtime/evaluate` (target < 50 ms), `/readyz` failures, HPA replica count, 421 rate	Prometheus → Grafana
Traces	OpenTelemetry is the intended tracing layer (PRD §57) — wire an OTel collector in the `monitoring` namespace	Tempo / X-Ray / Cloud Trace
Audit integrity	schedule `GET /v1/audit/verify` per org; alert on any failure	your alerting
Uptime	external check on `/healthz` (liveness) and `/readyz` (dependency health) per region	Pingdom / synthetics

9. Backups & disaster recovery

PRD §77 targets **RPO < 15 min, RTO < 1 h** per region.

- **Postgres** — automated backups + point-in-time recovery (managed service handles this). Test a restore into a scratch instance quarterly. This is the only store whose loss is unrecoverable.
- **ClickHouse** — replicated tables or scheduled `BACKUP` to object storage; event data is high-volume and reconstructable-ish but treat it as important.
- **Object store** — enable versioning + cross-AZ (or cross-region *within the same residency boundary*) replication.
- **Redis** — ephemeral by design (sessions, counters, cache). No backup needed; losing it logs everyone out and resets rate-limit windows.
- **Redpanda / Qdrant** — rebuildable; snapshot if convenient.
- **Secrets** — the per-region `AGENTGUARD_SECRET_KEY` must be in your secret manager's backup. Losing it invalidates every active session and API-key signature in that region.

Keep DR infrastructure for a region **inside that region's residency boundary**.

10. Security hardening checklist

- [ ] `AGENTGUARD_ENV=production`, `AGENTGUARD_OPEN_REGISTRATION=false`
- [ ] Unique `AGENTGUARD_SECRET_KEY` per region, from a CSPRNG, in a secret manager
- [ ] TLS everywhere at the edge; `rediss://` / TLS to Postgres if the link leaves the VPC
- [ ] `AGENTGUARD_CORS_ORIGINS` is an explicit allow-list — never `*`
- [ ] DB user is least-privilege (DML + DDL for migrations only, no superuser)
- [ ] NetworkPolicies default-deny; backing services not reachable from the internet
- [ ] Image scanning in CI; deploy by digest; non-root containers

- [ ] Secret manager rotation for DB / object-store / IdP credentials
- [ ] Rate-limit / WAF at the gateway in front of `api`
- [ ] `GET /v1/audit/verify` monitored per org
- [ ] MFA/SSO/SCIM secrets confirmed encrypted at rest (PRD §75)

---

11. CI/CD to production

`.github/workflows/ci.yml` already gates every PR: `policy-engine`, `sdk-python`, `sdk-typescript`, `sdk-dotnet`, `api` (Postgres + Redis services, `alembic check`, seed, pytest), `web` (typecheck + build), and `compose` (`docker compose config -q`).

A deploy pipeline on top of that:

1. On tag / merge to `main`: build + scan + push `agentguard-api` and `agentguard-web` by digest.
2. Apply the migrate `Job`; wait for success.
3. Roll the `api`, `ai-analyst`, `web` Deployments (rolling update, surge 1).
4. Post-deploy smoke test (§12).
5. Promote region by region — never all regions at once.

`alembic check` in CI must print *"No new upgrade operations detected"* — if a model changed without a migration, the build fails. Migrations are forward-only; a rollback is a new migration.

---

12. Go-live smoke test

Per region, after deploy:

```
BASE=https://us.api.agentguard.example

curl -fsS $BASE/healthz                    # {"status":"ok"}
curl -fsS $BASE/readyz                     # 200 + {"postgres":true,"redis":true,...}
curl -fsS $BASE/v1/regions                 # current region + full discovery map
curl -fsS $BASE/                           # banner: environment == "production"
```

Then in the dashboard: create an org (or sign in), register an agent, define one policy, and call `POST /v1/runtime/evaluate` with a matching tool call — confirm the decision and that an entry appears in the Audit Log with an intact hash chain.

---

13. Troubleshooting

Symptom	Cause / fix
Pods never become ready	`/readyz` needs Postgres **and** Redis — check the Secret's `DATABASE_URL` / `REDIS_URL` and NetworkPolicy egress
`alembic check` fails in CI	a model changed without a migration — `alembic revision --autogenerate` and commit it
Every API call returns 401	clock skew or wrong `AGENTGUARD_SECRET_KEY` between replicas — all `api` pods in a region must share one key
API returns `421 Misdirected Request`	the org is homed in another region — client should follow `X-AgentGuard-Region-Url`
Dashboard loads but all calls fail CORS	`AGENTGUARD_CORS_ORIGINS` doesn't list the dashboard's exact origin, or `NEXT_PUBLIC_API_BASE_URL` was baked wrong (rebuild `web`)
Analyst replies say `engine: fallback`	no `ANTHROPIC_API_KEY` — expected; deterministic router is in use
Migrate Job fails on ClickHouse	`CLICKHOUSE_HOST` unreachable — the Job needs Postgres **and** ClickHouse; check both before gating the rollout
p95 of `/v1/runtime/evaluate` creeping up	scale `api` (HPA target too high), check Redis latency, confirm the policy cache is warm

---

Generated source: `docs/DEPLOYMENT.md`. *See also* `RUNNING.md`, `ARCHITECTURE.md`, `MULTI_REGION.md`.